

# 操作系统课程设计

## 第二部分、并发 和 进程通信（经典IPC问题）

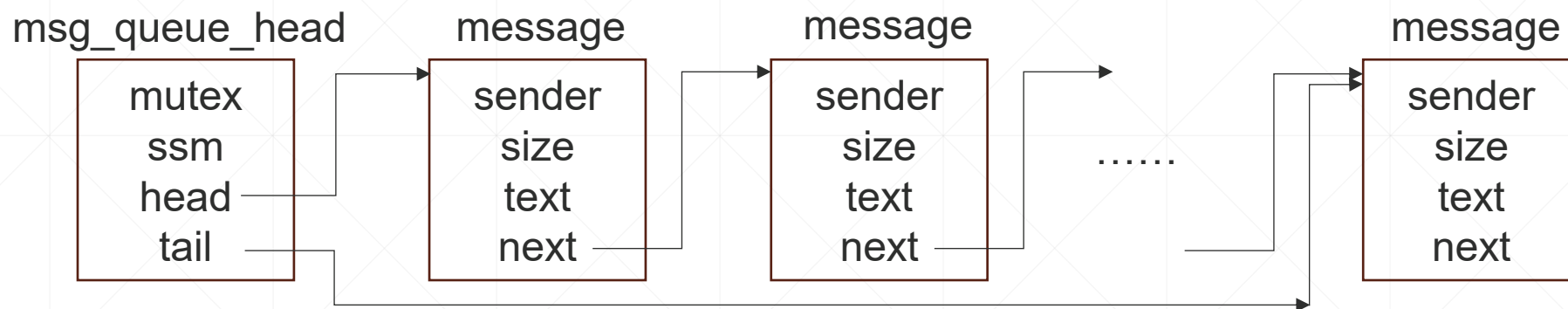
---

同济大学计算机系

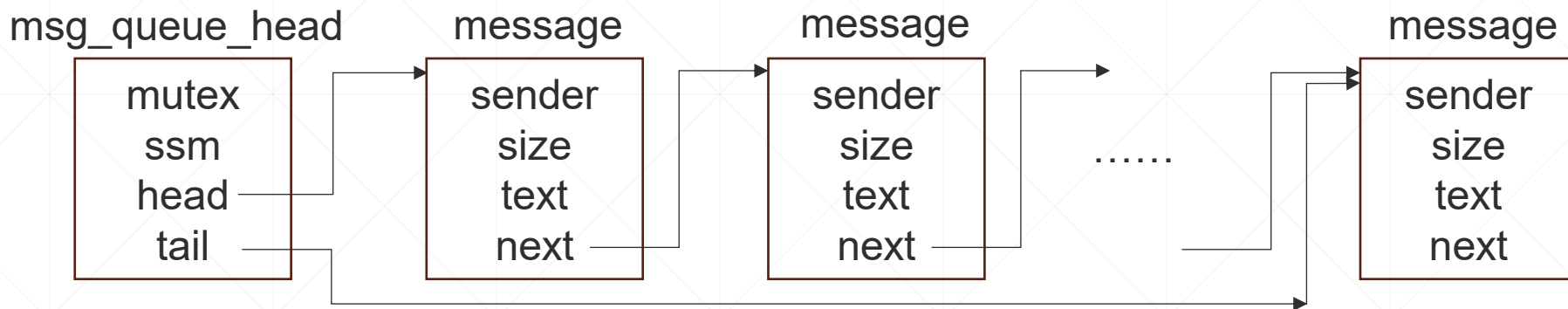


# Part 1

# 一、消息队列



- 服务器进程配消息队列，接纳客户请求。
- 客户机进程执行send( )操作向服务器提交请求。
- 服务器进程执行take( )操作从消息队列取消息。
- 消息队列FIFS。
  - 新请求放队尾。
  - 队首是服务器正在处理的消息。
  - 队首消息处理完毕，出列。服务器处理下一个队首消息。队列空，服务器进程入睡。



### send( )向消息队列发消息

```
semaphore mutex;
mutex.value = 1;
semaphore ssm;
ssm.value = 0;
```

- p(mutex) // 锁住消息队列
- 消息放队尾
- v(ssm) // 改变消息的数量、唤醒可能入睡的服务器进程
- v(mutex) // 解锁消息队列

### take( )从消息队列取消息

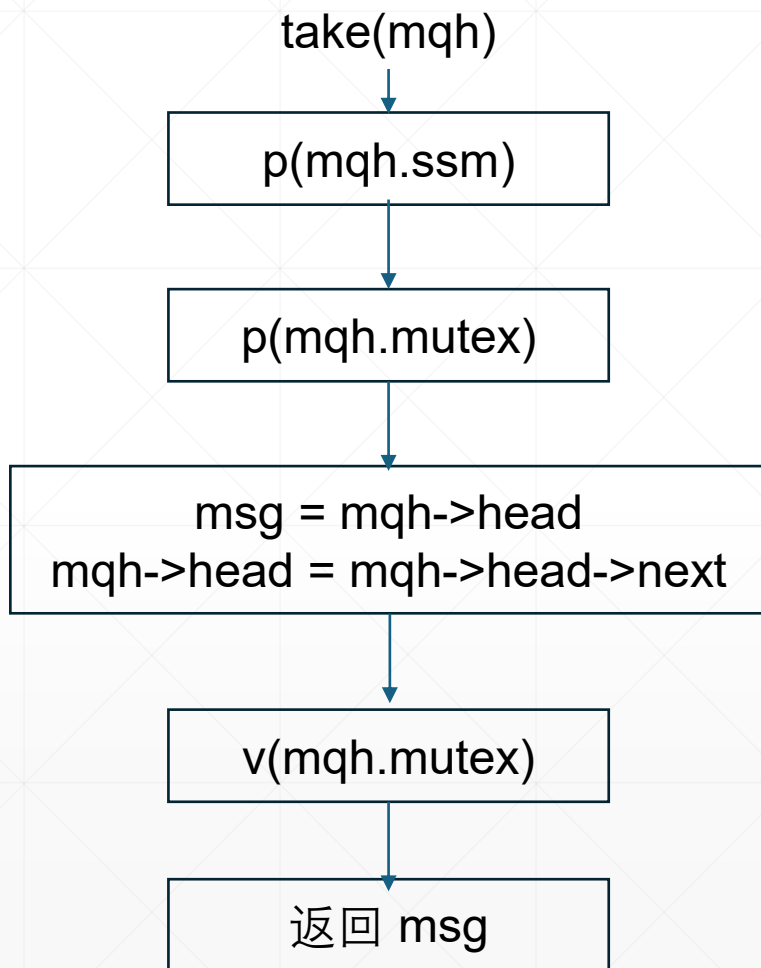
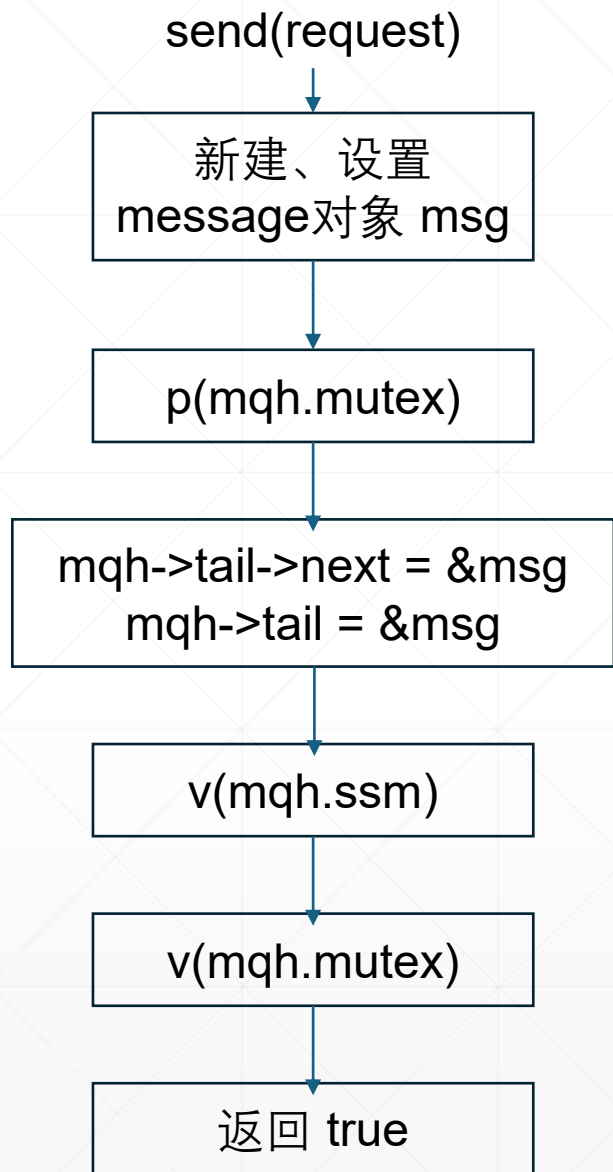
- p(ssm) // 队列为空时，入睡
- p(mutex) // 锁住消息队列
- 取队首消息 → m
- v(mutex) // 解锁消息队列
- 返回m

## 实现

```
semaphore mutex;  
mutex.value = 1;  
semaphore ssm;  
ssm.value = 0;
```

```
msg_queue_head mqh;
```

```
.....  
while( )  
{  
    request = take(mqh);  
    处理消息;  
    将处理结果返回给客户;  
    free(request);  
}  
.....
```



## 二、生产者、消费者问题

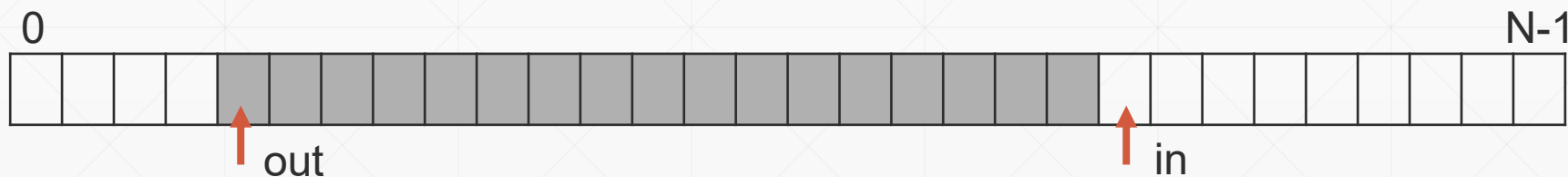
1. **生产者**进程和**消费者**进程用有界缓冲区传数据。有界缓冲区buffer有N个单元，每个单元可以放一个数据。要求：放入缓存区的每个数据，被处理一次，且仅被处理一次。 FIFS。

2. 生产者进程：产生一个数据，将其存入缓冲区（in 指针）；循环。

$\text{buffer}[\text{in}] = \text{item};$   
 $\text{in} = (\text{in} + 1) \bmod n;$

3. 消费者进程：从缓冲区取出一个数据（out 指针），处理；循环。

$\text{item} = \text{buffer}[\text{out}];$   
 $\text{out} = (\text{out} + 1) \bmod n;$

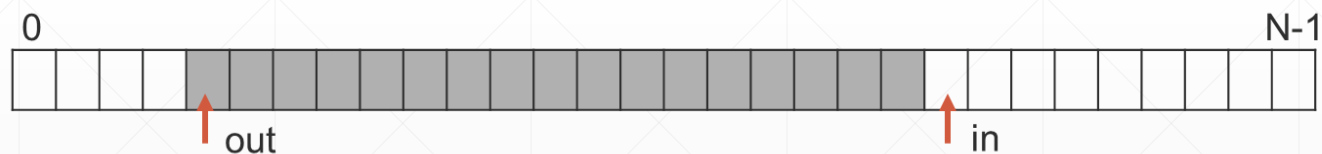
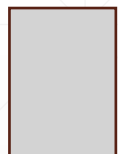


# 有界缓存的作用

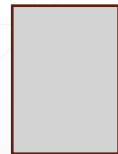
避免紧耦合。临时消除生产者进程和消费者进程的运行速度差异。

- 一段时间，生产者进程运行太快，没问题。只要还有空缓存块，它就能继续跑。
- 一段时间，消费者进程运行太快，没问题。只要缓存还有新数据，它就能继续跑。

产生数据



处理数据



同步条件和同步信号量设置：

- (1) 没有空缓存块，生产者进程入睡。 设置信号量`empty`，描述空缓存块数量。 `p(empty)`会让生产者进程入睡
- (2) 没有新数据，消费者进程入睡。 设置信号量`full`，描述新数据数量。 `p(full)`会让消费者进程入睡



# 一个生产者、一个消费者

同步信号量

full (初值为0) , 数据的数量

empty (初值为N) , 空缓存的数量

```
void producer()
```

```
{
```

```
    int item;
```

```
    while(true)
```

```
    {
```

```
        item = produceItem(); //产生一个数据
```

```
        p(empty); // 用掉一个空缓存。如果没有就睡觉
```

```
        buffer[in] = item; // 把数据放进缓存区
```

```
        in=(in+1) mod n;
```

```
        v(full); // 新数据++, 通知消费者进程
```

```
    }
```

```
}
```

```
void consumer()
```

```
{
```

```
    int item;
```

```
    while(true)
```

```
    {
```

```
        p(full);
```

```
        item=buffer[out];
```

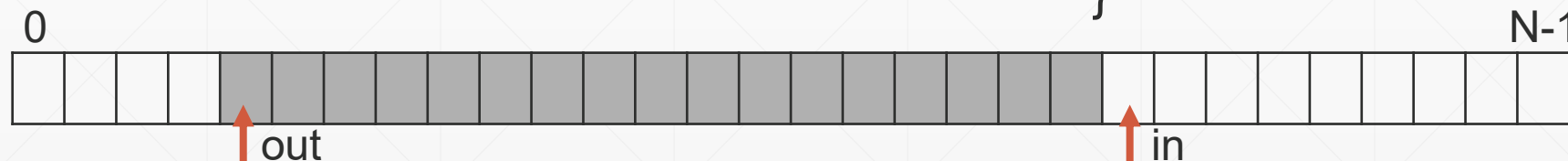
```
        out=(out+1) mod n;
```

```
        v(empty);
```

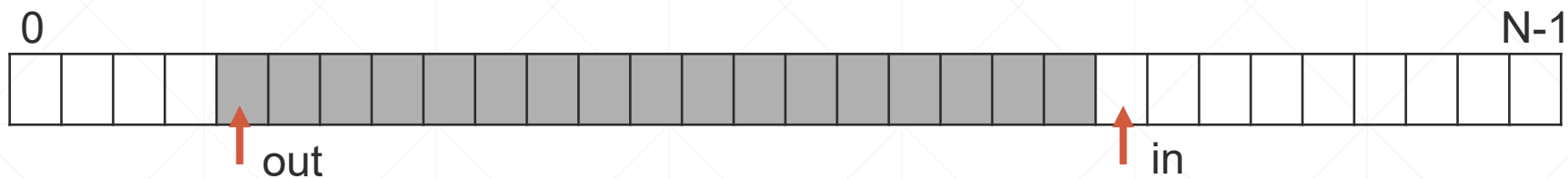
```
        consumeItem(item);
```

```
    }
```

```
}
```



## 这个解，没办法解决多生产者、多消费者问题



- 互斥没做好
  - 多个生产者同时访问 in 指针指向的单元，后写进的数据会覆盖先前数据，导致数据丢失。
  - 多个消费者同时访问 out 指针指向的单元，多次读取相同数据。
- 确保进程互斥访问缓存

# 多个生产者、多个消费者

# 正确的解

```
void producer()
{
    int item;
    while(true)
    {
        item = produceItem();
        p(empty);
        p(mutex);
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex);
        v(full);
    }
}
```

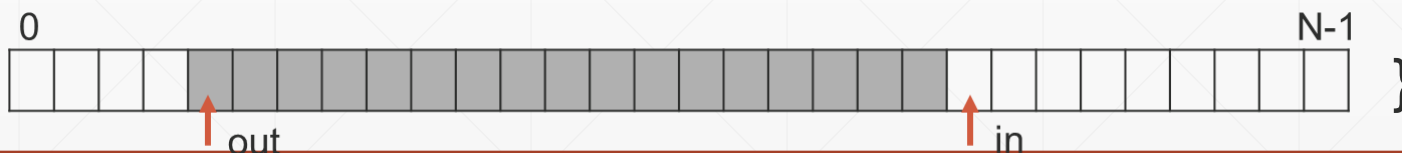
## 同步信号量

full (初值为0) , 数据的数量  
empty (初值为N) , 空缓存的数量

## 互斥信号量

mutex (初值为1) , 缓存池的访问权限

```
void consumer()
{
    int item;
    while(true)
    {
        p(full);
        p(mutex);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex);
        v(empty);
        consumeItem(item);
    }
}
```



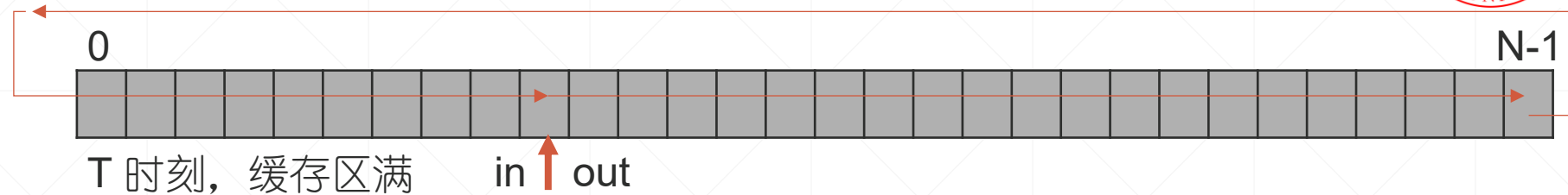
## 多个生产者、多个消费者

```
void producer()
{
    int item;
    while(true)
    {
        item = produceItem();
        p(empty);
        p(mutex);
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex);
        v(full);
    }
}
```

## 死锁

```
void consumer()
{
    int item;
    while(true)
    {
        p(full);
        p(mutex);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex);
        v(empty);
        consumeItem(item);
    }
}
```

# 产生死锁的情形 1



```
void producer()
{
    int item;
    while(true)
    {
        item = produceItem();
        p(mutex); // 1 拿到缓存区使用权
        p(empty); // 2 入睡等待空缓存
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex);
        v(full);
    }
}
```

无法释放缓存使用权

第2种情形, 请同学们自己考虑

```
void consumer()
{
    int item;
    while(true)
    {
        p(mutex); // 3 入睡等待缓存区使用权
        p(full);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex);
        v(empty);
        consumeItem(item);
    }
}
```

无法释放空缓存

死锁：并发系统中，因资源管理不当导致的进程彼此等待，均无法前进的现象。

## 另一个正确的解

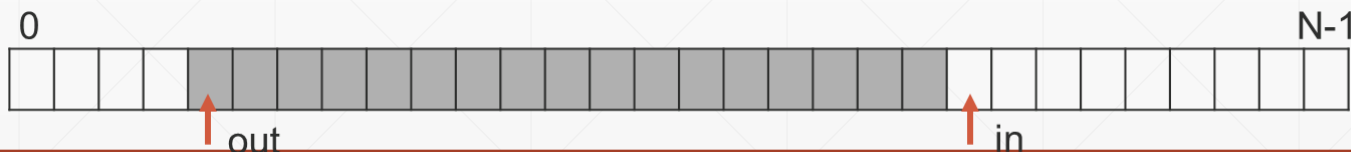
```
void producer()
{
    int item;
    while(true)
    {
        item = produceItem();
        p(empty);
        p(mutex1);
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex1);
        v(full);
    }
}
```

### 同步信号量

full (初值为0) , 数据的数量  
empty (初值为N) , 空缓存的数量

### 互斥信号量

mutex1 (初值为1) , 保护共享变量 in  
mutex2 (初值为1) , 保护共享变量 out



```
void consumer()
{
    int item;
    while(true)
    {
        p(full);
        p(mutex2);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex2);
        v(empty);
        consumeItem(item);
    }
}
```

## 三、读者/写者问题

- 读操作，不会改变共享变量的值。
- 写操作，改变共享变量的值。两种写操作：更新 和 赋值。
- 共享变量 需要 互斥访问。但是，读操作是可以并发的。允许读操作并发可以提高并发系统的效率。





# 状态分析

- 写操作必须互斥，不能与其它操作并发。用一个互斥信号量就能搞定。
- 读操作可以和读操作并发，不能与写操作并发。实现上，我们需要一个读者计数器。
  - 读者数量非0时，新的读操作可以并发，否则必需与写操作互斥。
  - 细节：读者计数器是一个共享变量，需要一个互斥信号量予以保护。

# 解

```
int  readcount = 0;
semaphore  wmutex = 1,  r = 1;
```

```
void writer()
{
    while(true)
    {
        p(wmutex); // 共享变量无人访问 ?
        WRITEUNIT(); // 更新
        v(wmutex); // 解锁
    }
}
```

```
void reader()
{
    while(true)
    {
        p(r);    // 互斥, 观察系统状态
        readcount++;
        if (readcount == 1) } 有读者吗 ?
            p(wmutex); 没有, 有写者吗 ? 有, 则入睡
        v(r);      无, 封锁写进程, 确保读到数据当前的值

        READUNIT(); 有。或无读者, 无写者。 执行读操作

        p(r);    // 离开前, 互斥修正系统状态。最后一个读者
        readcount--;
        if (readcount == 0)
            v(wmutex);
        v(r);
    }
}
```

## 这个解，读者有优先权 会饿死写进程

观察如下读写请求 R1,W1,R2,R3,R4,R5,R6。  
所有请求均匀到达系统，每秒1个，服务时间2秒1个。  
问（1）写请求W1何时得到处理（2）完成所有读写  
操作总耗时是多少？

```
void writer()
{
    while(true)
    {
        p(wmutex);
        WRITEUNIT();
        v(wmutex);
    }
}
```

```
void reader()
{
    while(true)
    {
        p(r);
        readcount++;
        if (readcount == 1)
            p(wmutex);
        v(r);

        READUNIT();

        p(r);
        readcount--;
        if (readcount == 0)
            v(wmutex);
        v(r);
    }
}
```

# 写进程优先的解决方案

```
int readcount = 0;
int writecount = 0;
semaphore wmutex, r, rmutex, w;
wmutex.value = 1;
r.value = 1;
rmutex.value = 1;
w.value = 1;
void writer()
{
    while(true) {
        p(w);
        writecount++;
        if(writecount == 1)
            p(rmutex);
        v(w);
        p(wmutex);
        WRITEUNIT();
        v(wmutex);
        p(w);
        writecount--;
        if(writecount == 0)
            v(rmutex);
        v(w);
    }
}
```

drong 20:31 回复 x

读操作执行时，第一个写进程用 p(rmutex) 阻止后续的读进程累加 readcount 计数器。写请求不会饿死。

drong

写操作执行时，后续到达的写请求会累加计数器writecount。读进程没有锁wmutex++操作，所以会饿死。

```
void reader()
{
    while(true) {
        p(rmutex);
        p(r);
        readcount++;
        if (readcount == 1)
            p(wmutex);
        v(r);
        v(rmutex);
        READUNIT();
        p(r);
        readcount--;
        if (readcount == 0)
            v(wmutex);
        v(r);
    }
}
```

drong 20:34 回复 x

信号量 rmutex 和 r 都是必需的。否则，会死锁。等着进入临界区的写进程（因wmutex信号量）用rmutex锁住读者进程，使其无法执行count--、释放wmutex信号量。

读操作执行期间，如果没有写请求，readcount会递增，多个读请求并发。

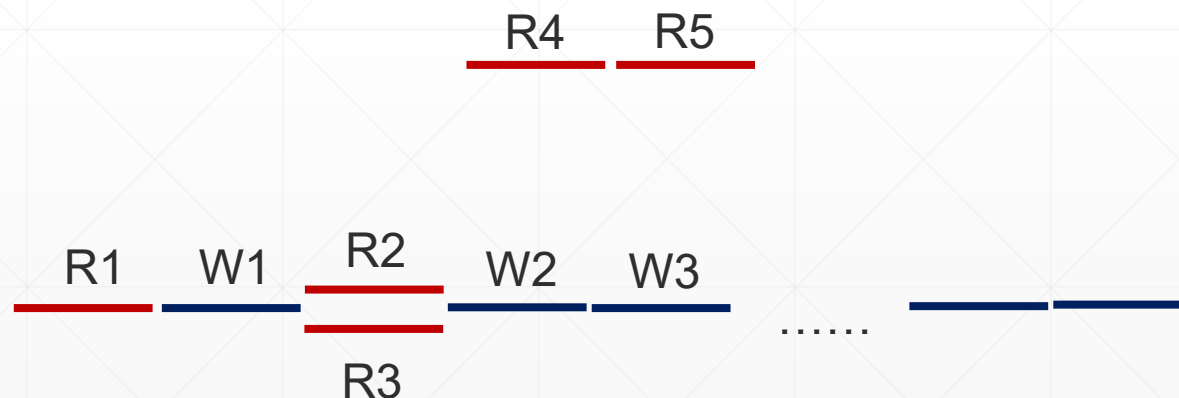
思考以下读写请求序列：

R1,W1,R2,R3,W2,W3,R4,R5,W4.....全是W请求



**观察如下读写请求 R1,W1,R2,R3,W2,W3,R4,R5,W4.....全是W**  
**看写请求怎样饿死读请求R4、R5。**

**假设：所有请求均匀到达系统，间隔1.1s；请求服务时间2s。**





# 读写进程公平的解

- **R1,W1,R2,R3,W2,W3,R4,R5,W4.....全是W请求。**
- 批处理，相邻的多个读请求组成一组，每个写请求是一组。先来先服务。
- 写伪代码。感受工程师设计系统时必须做的权衡。
- 再改善。

# 读写进程公平的解决方案

```
int readcount = 0 ;
int writecount = 0 ;

semaphore writemutex ;
writemutex.value = 1;
```

```
void writer( )
{
    while(true) {
        p(wmutex);
        p(w);
        writecount++;
        if( writecount == 1 )
            p(rmutex);
        v(w);
        v(wmutex);

        p(writemutex);
        writeunit( );
        v(writemutex);

        p(w);
        writecount--;
        if( writecount == 0 )
            v(rmutex);
        v(w);
    }
}
```

```
semaphore wmutex, w, rmutex, r;
wmutex.value = 1;
rmutex.value = 1;
w.value = 1;
r.value = 1;
```

```
void reader()
{
    while(true) {
        p(rmutex);
        p(r);
        readcount++;
        if( readcount == 1 )
            { p(wmutex); p(writemutex); }
        v(r);
        v(rmutex);

        readunit( );

        p(r);
        readcount--;
        if( readcount == 0 )
            { v(wmutex); v(writemutex); }
        v(r);
    }
}
```

思考以下读写请求序列:

R1,W1,R2,R3,W2,W3,R4,R5,W4.....全是W请求

## 习题:

- 小羊过河。 一条小河，一座独木桥。河东岸有一群小白羊，要去西岸吃草；河西岸有一群小黑羊，要去东岸吃草。用P、V操作解决这个问题，要求：在不饿死小羊的前提下，提高系统的并发效率。